

Automatic Temporal Human Action Detection Using a CLIP-Based Video Recognition System: From Prompt-Based Action Recognition to a User-Friendly Temporal Detection Prototype

Syed Md. Mahid, Hasna Hena, KM Tanvir Ahammad

Artificial Intelligence Course

Student IDs: 2022-2-50-026, 2022-3-50-007, 2022-2-50-024

Abstract—Human action recognition aims to identify what a person is doing in a video. In many practical videos, however, multiple actions occur sequentially, so identifying the action alone is not sufficient; the system should also estimate when each action occurs. This paper presents a compact implementation project that starts from a pretrained CLIP vision-language model and is inspired by the video-text matching idea of ActionCLIP. In Milestone 2, the system samples representative video frames and compares them with natural-language action prompts to perform zero-shot-style action recognition. A prompt comparison experiment showed that the wording of the text prompt affects the similarity score, with the prompt “a video of a person playing guitar” producing the highest score among the tested variants. In Milestone 3, the system was extended to automatic temporal action detection: a full video is divided into overlapping three-second windows with a two-second step, three representative frames are sampled from each window, and CLIP is used to predict the most likely action. The final prototype was additionally wrapped in a Gradio user interface so that a user can upload a video and receive action predictions and a timeline without manually executing notebook cells. On a 33.3-second controlled multi-action video, the system produced a 17-window temporal sequence covering swimming, motorcycle riding, guitar playing, and basketball playing. The results demonstrate a practical transition from manually selected action classification to automatic temporal action detection, while also exposing limitations caused by fixed windows and a predefined action vocabulary.

Index Terms—Action recognition, CLIP, ActionCLIP, vision-language models, temporal action detection, zero-shot recognition, video understanding, Gradio, human action recognition.

I. INTRODUCTION

Human action recognition is an important problem in computer vision because many applications require an AI system to understand what people are doing in videos. Examples include sports analysis, surveillance, robotics, human-computer interaction, and video indexing. A simple action classifier may answer a question such as “Is the person swimming?” for a selected clip, but continuous videos often contain several activities in sequence. A practical system should therefore identify the action and its temporal location.

The project presented in this paper was developed as a three-milestone academic implementation. The initial proposal focused on CLIP/ActionCLIP-style video-text matching for

human action recognition. The main goal was not to train a new large neural network, but to build a working prototype using publicly available pretrained models and then extend that prototype with a clear system-level modification.

The first stage, corresponding to Milestone 2, implemented a lightweight CLIP-based recognizer. Video frames were sampled, and text descriptions were compared against the visual content. The second stage, Milestone 3, removed manual temporal segmentation. Instead of telling the program where an action starts and ends, the program automatically scans the entire video through overlapping time windows and produces a prediction for each window. Finally, a Gradio interface was added in Google Colab to make the prototype easier to test and demonstrate.

The main contributions of this project are therefore:

- a lightweight CLIP-based implementation for prompt-based human action recognition;
- an automatic sliding-window temporal detection mechanism that removes manual segment boundaries;
- a simple Gradio interface that integrates video upload, analysis, result display, and temporal visualization; and
- an experimental analysis of the system on controlled multi-action videos, including the final four-action sequence of swimming, motorcycle riding, guitar playing, and basketball playing.

II. RELATED WORK

A. CLIP

CLIP (Contrastive Language–Image Pre-training) learns a shared representation for visual content and natural language by training on image-text pairs [1]. Its key property for this project is that an image can be compared directly with several textual descriptions without training a new classifier for every new label. For example, the same image can be compared with “a person swimming” and “a person playing guitar,” and the higher similarity score can be interpreted as the preferred action.

TABLE I: Prompt comparison for the guitar action.

Prompt	Score
“a person playing guitar”	27.3678
“a person is playing guitar”	27.8668
“a video of a person playing guitar”	28.7593
“a person performing the action of playing guitar”	27.0212

B. ActionCLIP

ActionCLIP extends the vision-language paradigm to video action recognition by treating action recognition as a video-text matching problem [2]. The original research system includes temporal video modeling and training procedures designed for action-recognition benchmarks. In this project, the full ActionCLIP training architecture was not reproduced. Instead, the implementation uses a pretrained CLIP model on sampled video frames and follows the core video-text matching concept as a practical prototype. This distinction is important because it prevents the project from overstating its relationship to the original research system.

III. MILESTONE 2: PROMPT-BASED ACTION RECOGNITION

A. Objective

The Milestone 2 objective was to implement a working action-recognition prototype using a pretrained vision-language model and to investigate whether the wording of natural-language prompts changes the recognition score.

B. Implementation Environment

The implementation was developed in Google Colab using a Tesla T4 GPU. PyTorch was available with CUDA support. The main software components were Python, PyTorch, Hugging Face Transformers, OpenCV, NumPy, Matplotlib, and the pretrained `openai/clip-vit-base-patch32` model.

C. Basic Pipeline

The Milestone 2 pipeline is shown conceptually as:

Video $\rightarrow$ Sample Frames $\rightarrow$ CLIP $\rightarrow$ Action Prompts $\rightarrow$ Similarity Scores $\rightarrow$ Prediction (1)

For a candidate action a , a textual prompt of the form

$$P(a) = \text{“a video of a person”} + a \quad (2)$$

was evaluated against sampled frames. The action with the largest mean similarity score was selected.

D. Prompt Engineering Experiment

A guitar video was used to compare four textual prompt variants. The measured CLIP similarity scores were:

The result demonstrated that prompt wording affects the CLIP similarity score. The highest-scoring template was therefore adopted for the later action classification experiments. These values are logits/similarity scores, not probabilities or benchmark accuracy.

TABLE II: Controlled Milestone 2 action-recognition result.

Actual Action	Predicted Action	Result
Swimming	Swimming	Correct
Riding a motorcycle	Riding a motorcycle	Correct
Playing guitar	Playing guitar	Correct

E. Controlled Action Recognition

The same approach was then applied to a controlled multi-action test video containing three main action segments. The model was given three candidate actions: swimming, riding a motorcycle, and playing guitar. The three manually selected segments were all classified correctly, giving 3/3 correct predictions for this small controlled test. This result was used as a prototype validation rather than as a large-scale benchmark accuracy.

IV. MILESTONE 3: AUTOMATIC TEMPORAL ACTION DETECTION

A. Motivation

The main limitation of Milestone 2 was manual temporal segmentation. The code required the user to specify action regions, for example, “swimming = 0–7 s” and “guitar = 18–26 s.” This meant that the system could classify an action only after the human had already supplied information about where that action occurred.

Milestone 3 changes this design by asking the system to scan the complete video automatically. The user supplies only a video; the program creates the time windows and classifies them.

B. Sliding-Window Strategy

The final configuration used a three-second window and a two-second step. For a 33.3-second video, this produced 17 windows:

$$W_i = [t_i, \min(t_i + 3, T)], \quad t_i = 2(i - 1), \quad (3)$$

where T is the video duration.

The overlap between consecutive windows helps preserve coverage around action boundaries. Three representative frames were sampled from every window to limit computation while still observing multiple moments inside the segment.

C. Window-Level Classification

For each sampled frame f and candidate action a , CLIP computes an image-text score $s(f, a)$. The frame scores are averaged within a window:

$$\bar{s}(W_i, a) = \frac{1}{3} \sum_{j=1}^3 s(f_{i,j}, a). \quad (4)$$

The predicted action is then

$$\hat{a}_i = \arg \max_{a \in A} \bar{s}(W_i, a), \quad (5)$$

where A is the candidate action set.

For the final test, the candidate set was:

$$A = \{\text{swimming, motorcycle, guitar, basketball}\}. \quad (6)$$

D. Temporal Timeline

The window predictions are plotted against time to form an automatic action timeline. Importantly, the system is not given the manual action boundaries. It infers a sequence of predictions directly from the continuous video.

V. USER INTERFACE IMPLEMENTATION

To make the system easier to test, a Gradio interface was added to the same Google Colab notebook. The interface contains an upload component, an analysis/submit button, an input-video preview, a detection-results panel, and an action-timeline plot.

The UI transforms the notebook pipeline into a simple application workflow:

- 1) upload a video;
- 2) click Submit;
- 3) read video duration and create automatic windows;
- 4) sample frames and run CLIP classification;
- 5) display the predicted action for each time window; and
- 6) visualize the temporal sequence in the timeline plot.

This addition is primarily a usability contribution. It does not change the underlying recognition model, but it makes repeated testing possible without rewriting or manually rerunning the processing code.

VI. EXPERIMENTAL RESULTS

A. Final Four-Action Test

The final tested video was 33.3 seconds long at 60 FPS. The system generated 17 overlapping windows. The final observed predictions were:

The sequence can be summarized approximately by the centers of the prediction windows as:

- 0–8 s: swimming;
- 8–16 s: riding a motorcycle;
- 16–29 s: playing guitar; and
- 29–33.3 s: playing basketball.

Because the windows overlap, these intervals represent the dominant prediction regions rather than exact frame-level action boundaries.

B. Milestone 2 versus Milestone 3

The main progression is summarized in Table IV.

C. Interpretation

The most important outcome is not a benchmark percentage but the successful conversion of a manually segmented prototype into an automatic temporal detection pipeline. The final run covered four visible action stages and produced the expected ordering: swimming, motorcycle riding, guitar playing, and basketball playing. The UI screenshot also demonstrates that the same system can be tested interactively on a new video.

VII. DISCUSSION

The project demonstrates that a pretrained vision-language model can be used as a practical building block for a small action-understanding system. The approach is attractive for a short academic project because it avoids training a new neural network and provides interpretable similarity scores from natural-language prompts.

However, the current system remains a prototype. First, the candidate action vocabulary is fixed manually; if a video contains an action outside the list, the system must still choose one of the known labels. Second, fixed-size overlapping windows can blur the exact transition point between actions. Third, sampling only three frames per window may miss short or subtle actions. Finally, the experiment uses controlled videos and a small number of action classes, so the results should not be interpreted as a large-scale benchmark evaluation.

The final basketball result also illustrates why the candidate-action list matters. In an earlier run, basketball was not part of the candidate set, so the model could not output basketball. Once “playing basketball” was added to the candidate actions, the final three windows were correctly classified as basketball in the final UI run. This is an important practical lesson: zero-shot-style matching still requires the desired label vocabulary to be supplied to the inference procedure.

VIII. LIMITATIONS AND FUTURE WORK

The following improvements would make the system more robust:

- add an “unknown/transition” class to handle ambiguous windows;
- use adaptive rather than fixed temporal windows;
- sample more frames or use an actual temporal video encoder;
- automatically expand or retrieve candidate action descriptions;
- evaluate the system on a standard benchmark such as UCF101 rather than only controlled generated videos; and
- compare the prototype against the full ActionCLIP architecture under the same evaluation protocol.

IX. CONCLUSION

This paper presented a combined implementation of Milestones 2 and 3 for a CLIP-based human action-recognition project. Milestone 2 established a working prompt-based action-recognition prototype and demonstrated that prompt wording affects similarity scores. Milestone 3 removed manual temporal segmentation by using overlapping sliding windows, sampled frames, and window-level CLIP classification to produce an automatic action timeline. A Gradio interface was then added so that a user can upload a new video and run the full pipeline without manually executing the underlying code.

The final controlled test demonstrated the sequence swimming, riding a motorcycle, playing guitar, and playing basketball across a 33.3-second video. The system therefore achieved the intended transition from selected-segment classification to

TABLE III: Final automatic temporal action-detection output.

Window	Prediction	Window	Prediction	Window	Prediction	Window	Prediction
1	Swimming	5	Motorcycle	9	Guitar	13	Guitar
2	Swimming	6	Motorcycle	10	Guitar	14	Guitar
3	Swimming	7	Motorcycle	11	Guitar	15	Basketball
4	Swimming	8	Motorcycle	12	Guitar	16	Basketball
17	Basketball						

TABLE IV: Comparison of the two milestones.

Milestone 2	Milestone 3
Manual action segment selection	Automatic sliding-window scan
Selected clip classification	Full-video temporal classification
One prompt per candidate action	Same classifier reused across all windows
Manual notebook workflow	Gradio upload-and-analyze workflow
Action result for selected segments	Action timeline over the complete video

automatic temporal action detection, while keeping the implementation simple enough for a course project and transparent enough to explain in a presentation.

PROJECT RESOURCES

- Tested video: https://drive.google.com/file/d/1OSVEv546_Xg6cUXi4kPUdKEEzkuqr_Ka/view?usp=sharing
- Project code and UI: <https://colab.research.google.com/drive/1nkFcvG8azvQXDNqAknv6GrlcoJCinCIJ?usp=sharing>

REFERENCES

- [1] A. Radford et al., “Learning Transferable Visual Models From Natural Language Supervision,” in *Proc. ICML*, 2021.
- [2] M. Wang, J. Xing, and Y. Liu, “ActionCLIP: A New Paradigm for Video Action Recognition,” 2021. [Online]. Available: <https://arxiv.org/abs/2109.08472>
- [3] “ActionCLIP,” GitHub repository. [Online]. Available: <https://github.com/sallymmx/ActionCLIP>
- [4] “openai/clip-vit-base-patch32,” Hugging Face model repository. [Online]. Available: <https://huggingface.co/openai/clip-vit-base-patch32>

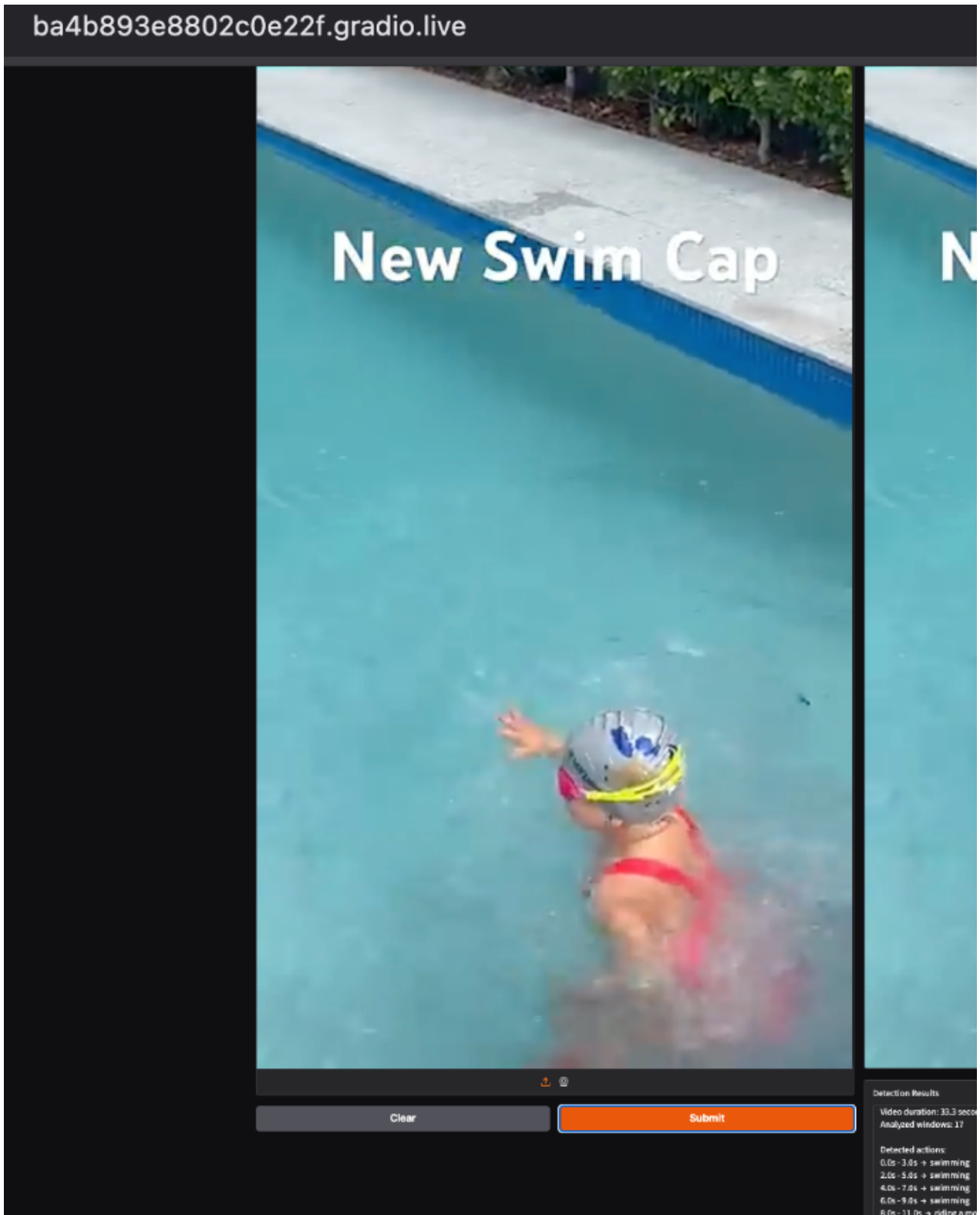
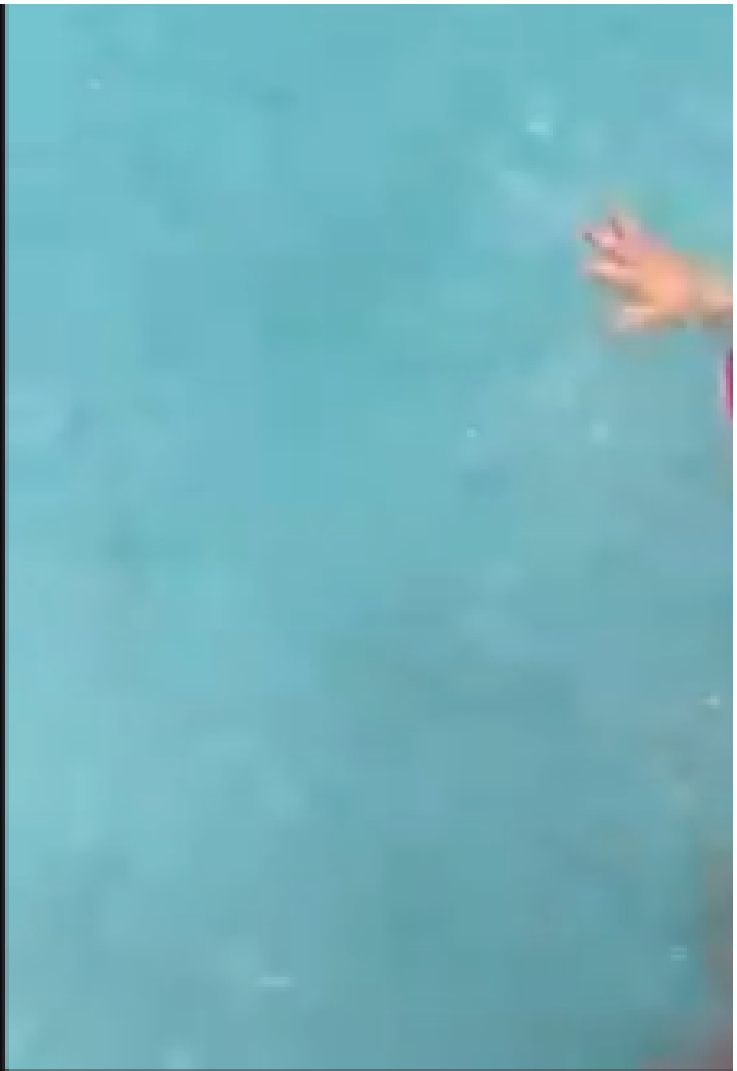


Fig. 1: Working Gradio user interface used to upload a video and display automatic action-detection results and the timeline.



Submit

Detection Results

Video duration: 33.3 seconds

Analyzed windows: 17

Detected actions:

0.0s - 3.0s + swimming
2.0s - 5.0s + swimming
4.0s - 7.0s + swimming
6.0s - 9.0s + swimming
8.0s - 11.0s + riding a motorcycle
10.0s - 13.0s + riding a motorcycle
12.0s - 15.0s + riding a motorcycle
14.0s - 17.0s + riding a motorcycle
16.0s - 19.0s + playing guitar
18.0s - 21.0s + playing guitar
20.0s - 23.0s + playing guitar
22.0s - 25.0s + playing guitar
24.0s - 27.0s + playing guitar
26.0s - 29.0s + playing guitar
28.0s - 31.0s + playing basketball
30.0s - 33.0s + playing basketball
32.0s - 33.3s + playing basketball

🔍 Action Timeline

Automatic Test

Basketball